

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет программной инженерии и компьютерной техники

Дисциплина:

«Исследование экосистем веб-языков и веб-технологий»

ОТЧЕТ ПО ПРОЕКТУ

«Песочница для сравнения производительности шаблонизаторов»

Выполнили:

Жидков Денис,
студент группы Р4109

(подпись)

Петрова Валентина,
студент группы Р4109

(подпись)

Шаулова Мария,
студент группы Р4109

(подпись)

Левченкова Наталья,
студент группы Р4107

(подпись)

Кильдеева Дарья, *студент группы Р4107*

(подпись)

Проверил:

Государев Илья Борисович, *доцент, факультет ПИиК*

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Целью проекта является разработка песочницы для сравнения производительности различных шаблонизаторов с возможностью сохранения полученных метрик.

Определены следующие задачи работы:

- Разработка пользовательского интерфейса для проведения эксперимента, поддерживающего ввод шаблонизатора из списка доступных, предоставления самого шаблона и данных для него, и для просмотра сохраненных данных и построение аналитического представления на их основе
- Разработка API поддерживающего сохранения результатов в базу данных
- Возможность автоматической генерации шаблонов для разных шаблонизаторов с использованием нейронных сетей на базе LLM

1. ОБЩИЕ ПОДХОДЫ К РАЗРАБОТКЕ И ЗОНЫ ОТВЕТСТВЕННОСТИ

Сервис разрабатывался с помощью контролируемого и документируемого AI-кодинга. Общий процесс реализации функциональности в рамках ответственности каждого из разработчиков можно описать так:

- формулирование задачи в виде текстового промпта с предоставлением необходимого для создания функциональности контекста в случае необходимости
- итеративное уточнение решения через диалог с ИИ
- документирование промежуточных решений и гипотез

Работа над проектом велась с использованием системы контроля версий Git и платформы GitHub с использованием следующего набора правил:

- основной веткой проекта является main
- разработка ведётся в отдельных ветках формата feature/*
- каждая задача связана с соответствующим issue
- используется единый стандарт именования веток и коммитов

Для автоматизации процессов сборки и проверки кода использовались GitHub Actions, включающие этапы установки зависимостей, линтинга, тестирования и сборки проекта.

С целью уменьшения пересечения разрабатываемых функциональностей, ускорения и организации командной работы сервис был разделен на следующие зоны ответственности:

- Шаулова Мария:
 - Проектирование архитектуры пользовательского интерфейса приложения
 - Разработка структуры и базовую реализацию ключевых страниц
 - Разработка дизайна пользовательского интерфейса, включая визуальную концепцию, макеты и пользовательские сценарии
 - Подготовка и структурирование общей части отчета по проекту
- Левченкова Наталья
 - Реализация страницы «Песочница» для ввода шаблонов и запуска экспериментов
 - Разработка пользовательских сценариев взаимодействия с системой
 - Интеграция клиент-серверного взаимодействия
 - Подготовка репозитория к разработке: формирование структуры проекта, создании задач (issues), настройке GitHub Actions и front/back pipeline

- Кильдеева Дарья
 - Реализация страницы «Результаты» для отображения данных экспериментов
 - Реализация визуализации результатов
 - Интеграция клиент-серверного взаимодействия
 - Подготовка репозитория к разработке: формирование структуры проекта, создании задач (issues), настройке GitHub Actions и front/back pipeline
- Петрова Валентина
 - Реализация компонентов интерфейса (модальные окна и вспомогательные элементы)
 - Настройка процессов сборки и деплоя проекта
 - Подготовила docker-окружение для фронтенда и бэкенд
 - Поддержка интеграции между частями сервиса
- Жидков Денис
 - Проектирование и реализации серверной части приложения
 - Разработка логики для реализации автоматической транспиляции шаблонов с помощью LLM
 - Интеграция клиент-серверного взаимодействия
 - Координация работы команды

Разработанная система представляет собой клиент-серверное веб-приложение, реализующее следующий ключевой сценарий:

1. Ввод пользователем данных эксперимента: шаблонизатор, шаблон и входные данные
2. Опциональная автоматическая генерация других шаблонов для других выбранных шаблонизаторов
3. Проведение компиляции шаблонов с измерением времени, затрачиваемого на преобразование
4. Сохранение результатов измерений в базе данных и последующее их сравнение в разделе результатов

Изображения интерфейса сервиса представлены на рисунках 1-3. Код проекта можно исследовать на: <https://github.com/dasha041404/perf-sandbox>

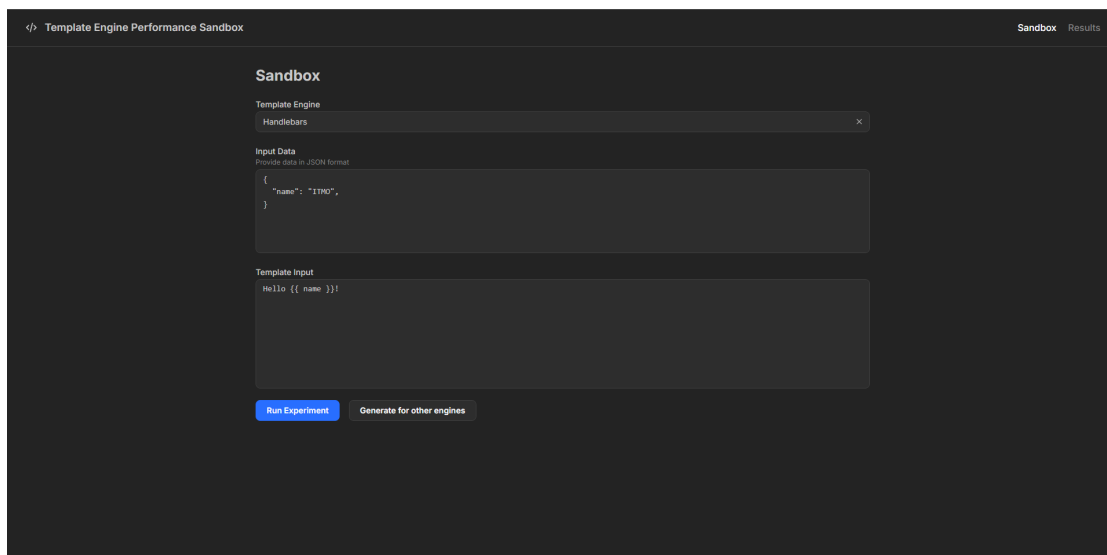


Рис. 1. Страница «Эксперимент»

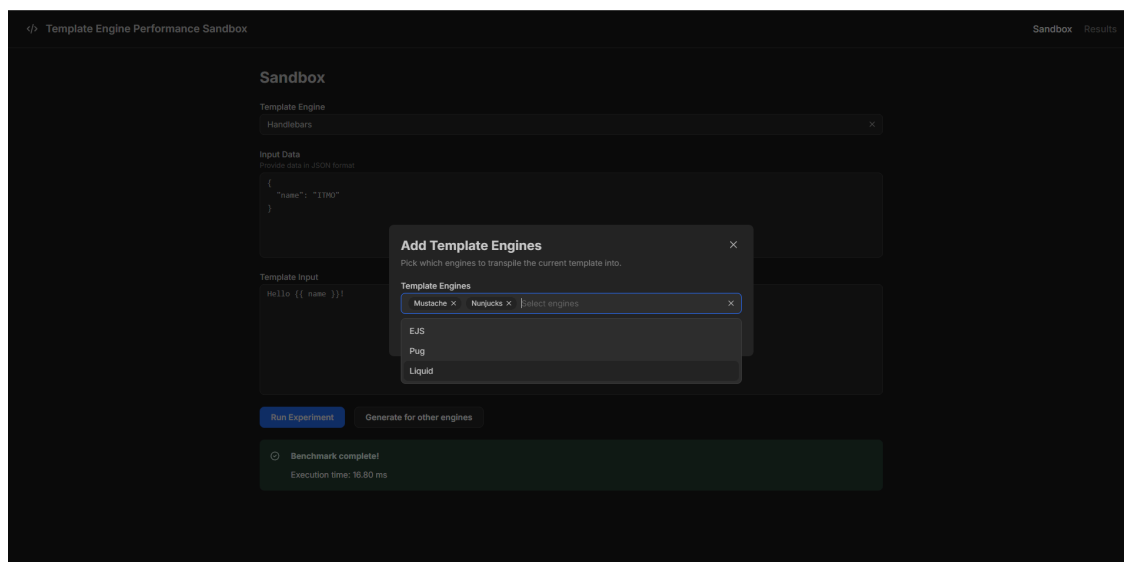


Рис. 2. Процесс автогенерации шаблонов для других шаблонизаторов

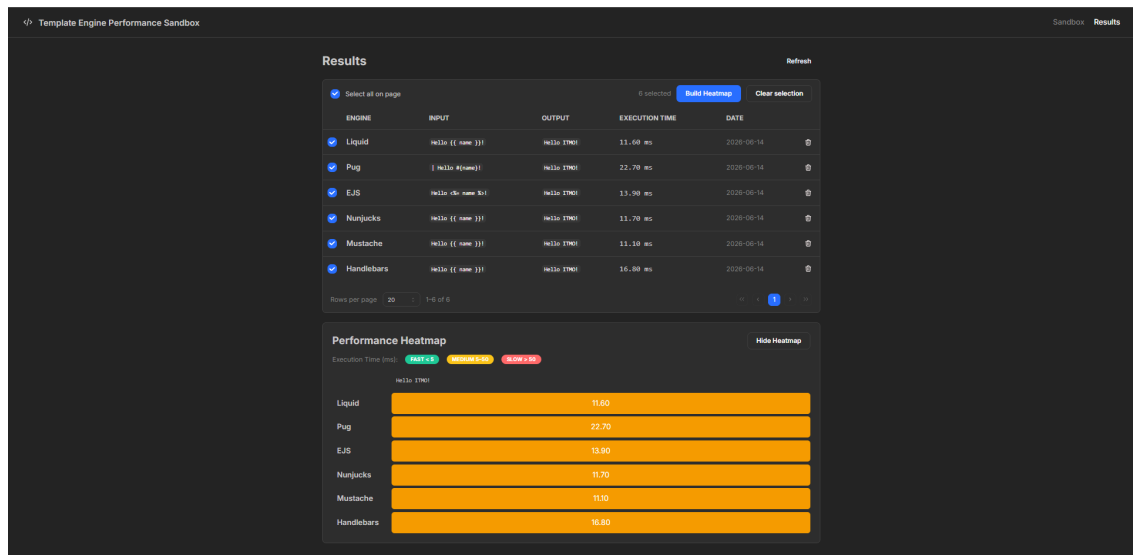


Рис. 2.Страница «Результаты»

2. РАЗРАБОТКА ИНФРАСТРУКТУРНОЙ ЧАСТИ ПРОЕКТА

В рамках проекта моя зона ответственности заключалась в подготовке инфраструктуры клиентской части приложения, настройке процессов разработки и организации взаимодействия между клиентом и сервером. Для реализации поставленных задач использовался GitHub Copilot в среде Visual Studio Code.

Работа была разбита на несколько этапов.

В рамках первого этапа выполнялась инициализация проекта и подготовка базовой структуры клиентского приложения. Необходимо было создать основу для дальнейшей разработки, настроить сборку проекта и организовать структуру каталогов. Для реализации данного этапа использовались промпты, представленные в листинге 1.

Листинг 1. Промпты первого этапа

Создай новый frontend-проект на React + TypeScript с использованием Vite.

Настрой структуру каталогов для дальнейшей разработки приложения.

Добавь конфигурацию маршрутизации и подготовь базовую структуру страниц.

Настрой npm-скрипты для локального запуска и production-сборки.

Проверь успешность запуска проекта и корректность сборки.

Подготовь README.md с инструкцией по запуску проекта, установке зависимостей и основным командам разработки.

В рамках второго этапа требовалось реализовать API-слой приложения для взаимодействия с серверной частью системы. В результате был сформирован отдельный сервисный слой, содержащий типизацию запросов и обработку ответов. Промпты данного этапа представлены в листинге 2.

Листинг 2. Промпты второго этапа

Необходимо реализовать клиентский слой для работы с backend API.

Создай отдельный модуль api-client для выполнения HTTP-запросов.

Добавь поддержку query-параметров, типизированных ответов и обработки ошибок.

Используй TypeScript для описания контрактов запросов и ответов.

Создай отдельные сервисы для работы с экспериментами, генерацией шаблонов и рендерингом.

Минимизируй дублирование логики между запросами.

В рамках третьего этапа была выполнена настройка процессов контроля качества кода и автоматизации локальной разработки. Для этого использовались ESLint, Prettier, Husky и lint-staged. Промпты данного этапа представлены в листинге 3.

Листинг 3. Промпты третьего этапа

Настрой ESLint для React и TypeScript проекта.

Добавь команды проверки и автоматического исправления ошибок.

Проверь конфигурацию на существующем коде проекта.

Настрой Prettier и единый стиль форматирования проекта.

Обеспечь совместную работу ESLint и Prettier без конфликтов конфигураций.

Настрой Husky и lint-staged.

Перед выполнением коммита автоматически запускай форматирование и проверку изменённых файлов.

Проверь работу Git Hooks локально.

В рамках четвертого этапа выполнялась контейнеризация приложения. Для этого были подготовлены Docker-конфигурации и обеспечена возможность запуска проекта в контейнерном окружении. Использованные промпты представлены в листинге 4.

Листинг 4. Промпты четвертого этапа

Создай Dockerfile для frontend-приложения на базе Node.js.

Настрой установку зависимостей и запуск приложения внутри контейнера.

Обеспечь возможность локальной разработки через Docker.

Проверь конфигурацию Docker.

Убедись, что контейнер корректно собирается и приложение доступно после запуска.

Использование GitHub Copilot позволило ускорить подготовку инфраструктурной части проекта, сократить время на создание типовых конфигураций и сосредоточиться на проверке и адаптации решений под требования проекта

ЗАКЛЮЧЕНИЕ

Результатом выполнения проекта является система, позволяющая проводить эксперименты для сравнения производительности шаблонизаторов, сохранять результаты и визуализировать их в удобной форме.

В ходе проекта продемонстрированы применимость ai-кодинга в командной разработке и получен практический опыт интеграции ИИ в процесс разработки ПО.